Final Results Notebook

This notebook supports the Final Results and Project Report by documenting the study design, model development, evaluation, and key findings for immigrant and non-immigrant employment outcomes.

Research Design and Data Preparation

In [109…

```
# 1. Import Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split, StratifiedKFold,
from sklearn.base import clone
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingC
```

In [110…

```
# 2. Load Dataset

file_path = "../data/raw/LFS_98100442.csv"
df = pd.read_csv(file_path)

print("Original dataset shape:", df.shape)
display(df.head())
```

```
Original dataset shape: (25401600, 32)
```

	REF_DATE	GEO	DGUID	Gender (3)	Age (15A)	Labour force status (8)	Major field of study - Classification of Instructional Programs (CIP) 2021 (63)	Lo of
0	2021	Canada	2021A000011124	Total - Gender	Total - Age	Total - Labour force status	Total - Major field of study - Classification ...	Lo o
1	2021	Canada	2021A000011124	Total - Gender	Total - Age	Total - Labour force status	Total - Major field of study - Classification ...	Lo o
2	2021	Canada	2021A000011124	Total - Gender	Total - Age	Total - Labour force status	Total - Major field of study - Classification ...	Lo o
3	2021	Canada	2021A000011124	Total - Gender	Total - Age	Total - Labour force status	Total - Major field of study - Classification ...	Lo o
4	2021	Canada	2021A000011124	Total - Gender	Total - Age	Total - Labour force status	Total - Major field of study - Classification ...	Lo o

5 rows × 32 columns

In [111…

```
# 3. Review Columns

print("Columns:")
for col in df.columns:
    print(col)
```

```
Columns:
REF_DATE
GEO
DGUID
Gender (3)
Age (15A)
Labour force status (8)
Major field of study - Classification of Instructional Programs (CIP) 2
021 (63)
Location of study (5)
Highest certificate, diploma or degree (16)
Coordinate
Immigrant status and period of immigration (11):Total - Immigrant statu
s and period of immigration[1]
Symbol
Immigrant status and period of immigration (11):Non-immigrants[2]
Symbol.1
Immigrant status and period of immigration (11):Immigrants[3]
Symbol.2
Immigrant status and period of immigration (11):Before 1980[4]
Symbol.3
Immigrant status and period of immigration (11):1980 to 1990[5]
Symbol.4
Immigrant status and period of immigration (11):1991 to 2000[6]
Symbol.5
Immigrant status and period of immigration (11):2001 to 2010[7]
Symbol.6
Immigrant status and period of immigration (11):2011 to 2021[8]
Symbol.7
Immigrant status and period of immigration (11):2011 to 2015[9]
Symbol.8
Immigrant status and period of immigration (11):2016 to 2021[10]
Symbol.9
Immigrant status and period of immigration (11):Non-permanent resident
s[11]
Symbol.10
```

Methodology: Data Cleaning and Preprocessing

In [112…

```
# 4. Data Cleaning and Preprocessing

df_clean = df.copy()

# Keep only employed/unemployed observations for modeling
df_clean = df_clean[df_clean["Labour force status (8)"].isin(["Employe

# Remove "Total" categories from key predictors where possible
filters = {
    "Gender (3)": "Total - Gender",
    "Age (15A)": "Total - Age",
    "Highest certificate, diploma or degree (16)": "Total - Highest ce
    "Location of study (5)": "Total - Location of study",
```

```
    "Major field of study - Classification of Instructional Programs (
}

for col, total_value in filters.items():
    if col in df_clean.columns:
        df_clean = df_clean[df_clean[col] != total_value].copy()

print("Cleaned dataset shape:", df_clean.shape)
display(df_clean.head())
```

```
Cleaned dataset shape: (2916480, 32)
```

	REF_DATE	GEO	DGUID	Gender (3)	Age (15A)	Labour force status (8)	Major fie st Classific Instruct Prog (CIP)
655297	2021	Canada	2021A000011124	Men+	15 to 24 years	Employed	postseco certif diplo d
655298	2021	Canada	2021A000011124	Men+	15 to 24 years	Employed	postseco certif diplo d
655299	2021	Canada	2021A000011124	Men+	15 to 24 years	Employed	postseco certif diplo d
655300	2021	Canada	2021A000011124	Men+	15 to 24 years	Employed	postseco certif diplo d
655301	2021	Canada	2021A000011124	Men+	15 to 24 years	Employed	postseco certif diplo d

5 rows × 32 columns

In [113…

```
# 5. Create Target Variable
```

```
df_clean["target"] = df_clean["Labour force status (8)"].map({
    "Employed": 1,
    "Unemployed": 0
})

print(df_clean["target"].value_counts(dropna=False))
```

```
target
1    1458240
0    1458240
Name: count, dtype: int64
```

Methodology: Feature Selection

In [114…

```
# 6. Select Features for Modeling

# These are more meaningful than only using the immigrant count column
candidate_features = [
    "REF_DATE",
    "GEO",
    "Gender (3)",
    "Age (15A)",
    "Highest certificate, diploma or degree (16)",
    "Location of study (5)",
    "Major field of study - Classification of Instructional Programs (
]

# Keep only columns that exist
features = [col for col in candidate_features if col in df_clean.colum

df_model = df_clean[features + ["target"]].copy()

print("Modeling dataset shape:", df_model.shape)
display(df_model.head())
```

```
Modeling dataset shape: (2916480, 8)
```

	REF_DATE	GEO	Gender (3)	Age (15A)	Highest certificate, diploma or degree (16)	Location of study (5)	Major Classi Instr Pr (CI
655297	2021	Canada	Men+	15 to 24 years	No certificate, diploma or degree	No postsecondary certificate, diploma or degree	postse ce di
655298	2021	Canada	Men+	15 to 24 years	High (secondary) school diploma or equivalency...	No postsecondary certificate, diploma or degree	postse ce di
655299	2021	Canada	Men+	15 to 24 years	Postsecondary certificate, diploma or degree	No postsecondary certificate, diploma or degree	postse ce di
655300	2021	Canada	Men+	15 to 24 years	Postsecondary certificate or diploma below bac...	No postsecondary certificate, diploma or degree	postse ce di
655301	2021	Canada	Men+	15 to 24 years	Apprenticeship or trades certificate or diploma	No postsecondary certificate, diploma or degree	postse ce di

Methodology: Train-Test Split

In [115…

```
# 7. Split Features and Target

X = df_model.drop(columns=["target"])
y = df_model["target"]

categorical_features = X.select_dtypes(include=["object"]).columns.tol
numeric_features = X.select_dtypes(include=["number"]).columns.tolist(

print("Categorical features:", categorical_features)
print("Numeric features:", numeric_features)
```

```
Categorical features: ['GEO', 'Gender (3)', 'Age (15A)', 'Highest certi
ficate, diploma or degree (16)', 'Location of study (5)', 'Major field
of study - Classification of Instructional Programs (CIP) 2021 (63)']
Numeric features: ['REF_DATE']
```

In [116…

```
# 8. Preprocessing

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
```

In [117…

```
# 9. Train-Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
X_train shape: (2333184, 7)
X_test shape: (583296, 7)
```

Model Development and Runtime Constraints

In [118…

```
# 10. Define Models and Runtime Settings

# To match the assignment rubric efficiently, the notebook keeps two m
# a linear baseline (Logistic Regression) and a nonlinear ensemble (Ra
FULL_DATA_MODELS = set()
MODEL_SAMPLE_SIZES = {
    "Logistic Regression": 300_000,
    "Random Forest": 100_000
}
CV_SAMPLE_SIZE = 30_000
```

```
CV_SPLITS = 3

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_st
    "Random Forest": RandomForestClassifier(
        n_estimators=50,
        max_depth=10,
        random_state=42,
        n_jobs=-1
    )
}
```

Model Evaluation: Effectiveness and Efficiency

In [119…

```
# 11a. Training Helpers

def get_training_data(model_name, X_data, y_data):
    if model_name in FULL_DATA_MODELS:
        return X_data, y_data, "full training set"

    sample_size = min(MODEL_SAMPLE_SIZES[model_name], len(X_data))
    X_sample, _, y_sample, _ = train_test_split(
        X_data,
        y_data,
        train_size=sample_size,
        random_state=42,
        stratify=y_data
    )
    return X_sample, y_sample, f"stratified sample of {sample_size:,}

results = []
trained_pipelines = {}

def train_and_store_model(model_name):
    model = models[model_name]
    print(f"\n--- {model_name} ---")

    X_fit, y_fit, training_note = get_training_data(model_name, X_trai
    print(f"Training data: {training_note}")

    pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("classifier", clone(model))
    ])

    start_train = time.time()
    pipeline.fit(X_fit, y_fit)
    end_train = time.time()
    train_time = end_train - start_train

    start_test = time.time()
    y_pred = pipeline.predict(X_test)
```

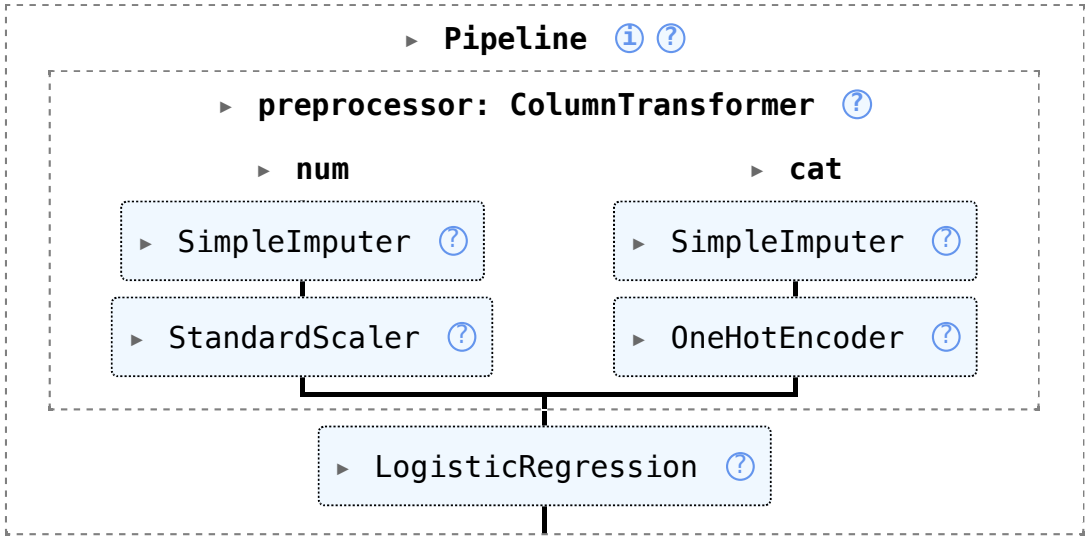
```
    end_test = time.time()
    test_time = end_test - start_test

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    print(f"Accuracy:  {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall:    {rec:.4f}")
    print(f"F1 Score:  {f1:.4f}")
    print(f"Train Time (sec): {train_time:.4f}")
    print(f"Test Time (sec):  {test_time:.4f}")

    print("\nClassification Report:")
    print(classification_report(y_test, y_pred, zero_division=0))

    results[:] = [row for row in results if row["Model"] != model_name
    results.append({
        "Model": model_name,
        "Training Rows": len(X_fit),
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1 Score": f1,
        "Train Time (sec)": train_time,
        "Test Time (sec)": test_time
    })

    trained_pipelines[model_name] = pipeline
    return pipeline
```

In [120…

```
# 11b. Train Logistic Regression

train_and_store_model("Logistic Regression")
```

```
--- Logistic Regression ---
Training data: stratified sample of 300,000 rows
Accuracy:  0.4995
Precision: 0.4995
Recall:    0.5175
F1 Score:  0.5083
Train Time (sec): 0.6026
Test Time (sec):  0.5018

Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.48      0.49    291648
           1       0.50      0.52      0.51    291648

    accuracy                           0.50    583296
   macro avg       0.50      0.50      0.50    583296
weighted avg       0.50      0.50      0.50    583296
```

Out[120…

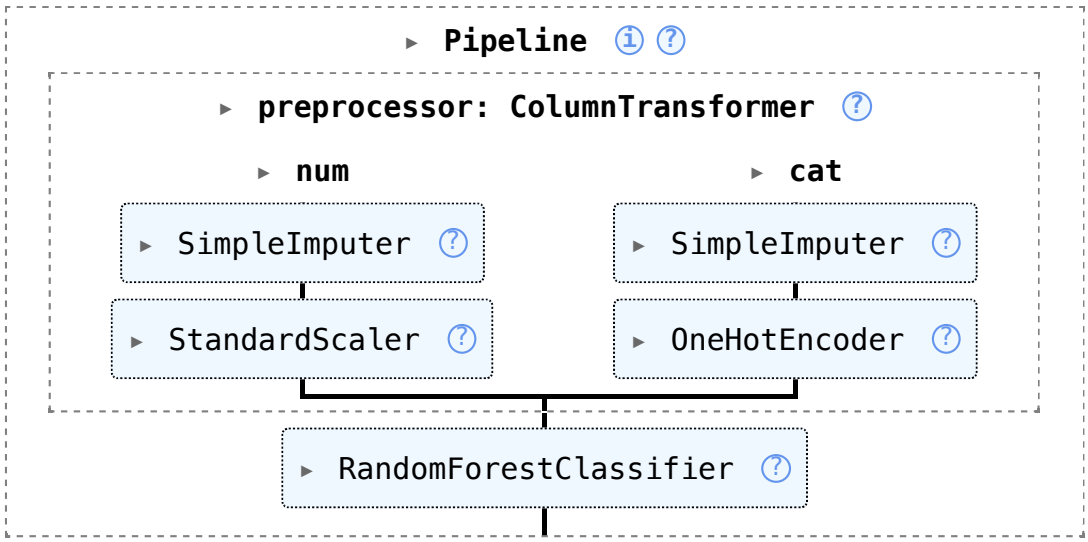


In [121…

```python
# 11c. Train Random Forest

train_and_store_model("Random Forest")
```

```
--- Random Forest ---
Training data: stratified sample of 100,000 rows
Accuracy:  0.4960
Precision: 0.4964
Recall:    0.5496
F1 Score:  0.5216
Train Time (sec): 0.7192
Test Time (sec):  0.7840

Classification Report:
              precision    recall  f1-score   support

           0       0.50      0.44      0.47    291648
           1       0.50      0.55      0.52    291648

    accuracy                           0.50    583296
   macro avg       0.50      0.50      0.49    583296
weighted avg       0.50      0.50      0.49    583296
```

Out[121…



Gradient Boosting was removed to keep the workflow computationally efficient while still meeting the rubric requirement for multiple models and comparative evaluation.

Model Evaluation Summary

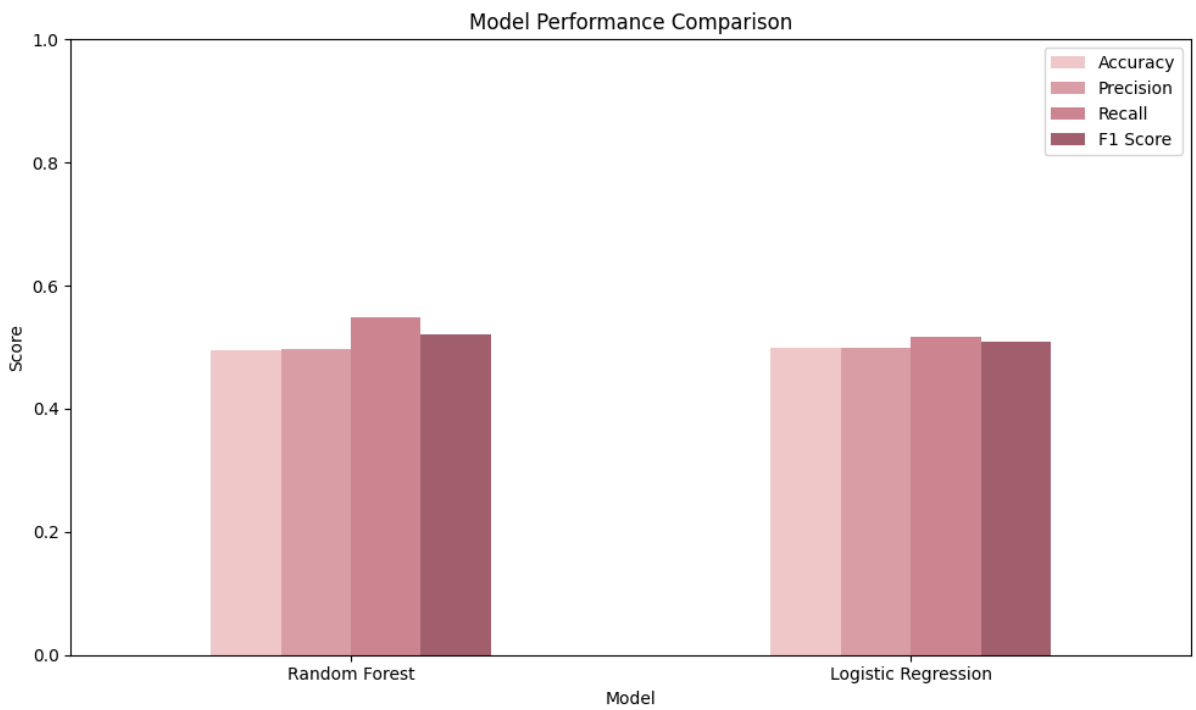
In [122…

```
# 12. Comparison Table

results_df = pd.DataFrame(results).sort_values(by="F1 Score", ascendin
display(results_df[["Model", "Training Rows", "Accuracy", "Precision",
```

	Model	Training Rows	Accuracy	Precision	Recall	F1 Score	Train Time (sec)	T (
1	Random Forest	100000	0.496026	0.496410	0.549573	0.521641	0.719161	0.784
0	Logistic Regression	300000	0.499496	0.499513	0.517466	0.508331	0.602599	0.50

In [123…

```
# 13. Visualize Model Performance

results_plot = results_df.set_index("Model")[["Accuracy", "Precision",
results_plot.plot(kind="bar", figsize=(10, 6), color=["#efc7c9", "#d99
plt.title("Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```

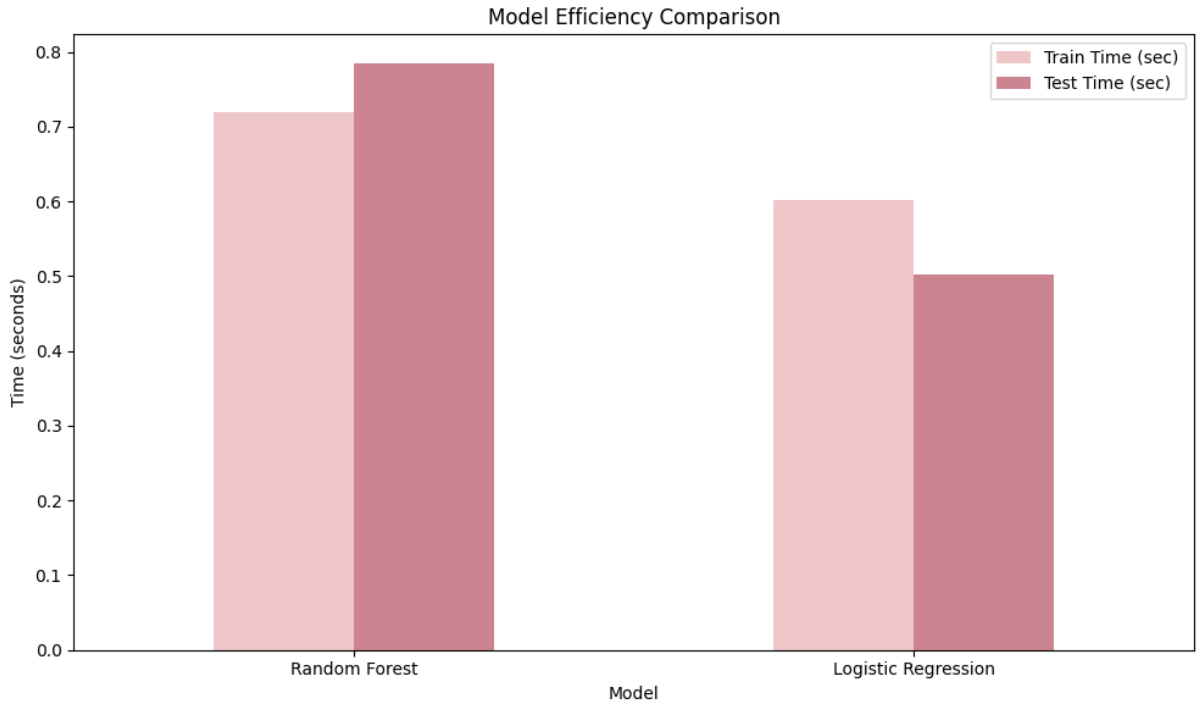


Efficiency Analysis

In [124…

```
# 14. Visualize Model Efficiency

efficiency_plot = results_df.set_index("Model")[["Train Time (sec)", "
efficiency_plot.plot(kind="bar", figsize=(10, 6), color=["#efc7c9", "#
plt.title("Model Efficiency Comparison")
plt.ylabel("Time (seconds)")
plt.xticks(rotation=0)
plt.tight_layout()
```

```
plt.show()
```



Stability and Robustness

In [125…

```
# 15a. Cross-Validation Helpers

# Cross-validation is run on a smaller stratified sample to document m
# without making the notebook too slow to reproduce.
cv = StratifiedKFold(n_splits=CV_SPLITS, shuffle=True, random_state=42

cv_sample_size = min(CV_SAMPLE_SIZE, len(X))
X_cv, _, y_cv, _ = train_test_split(
    X,
    y,
    train_size=cv_sample_size,
    random_state=42,
    stratify=y
)

print(f"Cross-validation base sample size: {len(X_cv):,} rows")

cv_results = []

def run_cv_for_model(model_name):
    model = models[model_name]
    model_cv_size = min(MODEL_SAMPLE_SIZES[model_name], len(X_cv))

    if model_cv_size == len(X_cv):
        X_cv_model, y_cv_model = X_cv, y_cv
    else:
        X_cv_model, _, y_cv_model, _ = train_test_split(
            X_cv,
```

```
            y_cv,
            train_size=model_cv_size,
            random_state=42,
            stratify=y_cv
        )

    print(f"Running CV for {model_name} on {len(X_cv_model):,} rows")

    pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("classifier", clone(model))
    ])

    scores = cross_val_score(
        pipeline,
        X_cv_model,
        y_cv_model,
        cv=cv,
        scoring="f1",
        n_jobs=None
    )

    cv_results[:] = [row for row in cv_results if row["Model"] != mode
    cv_results.append({
        "Model": model_name,
        "CV Rows": len(X_cv_model),
        "CV Mean F1": scores.mean(),
        "CV Std F1": scores.std(),
        "Fold Scores": scores
    })

    return scores
```

```
Cross-validation base sample size: 30,000 rows
```

In [126…

```
# 15b. Cross-Validate Logistic Regression

run_cv_for_model("Logistic Regression")
```

```
Running CV for Logistic Regression on 30,000 rows
```

Out[126…

```
array([0.50161225, 0.49382716, 0.49546225])
```

In [127…

```
# 15c. Cross-Validate Random Forest

run_cv_for_model("Random Forest")
```

```
Running CV for Random Forest on 30,000 rows
```

Out[127…

```
array([0.49624361, 0.48121848, 0.51955254])
```

Only Logistic Regression and Random Forest are cross-validated because they are the two models retained for final comparison and interpretation.

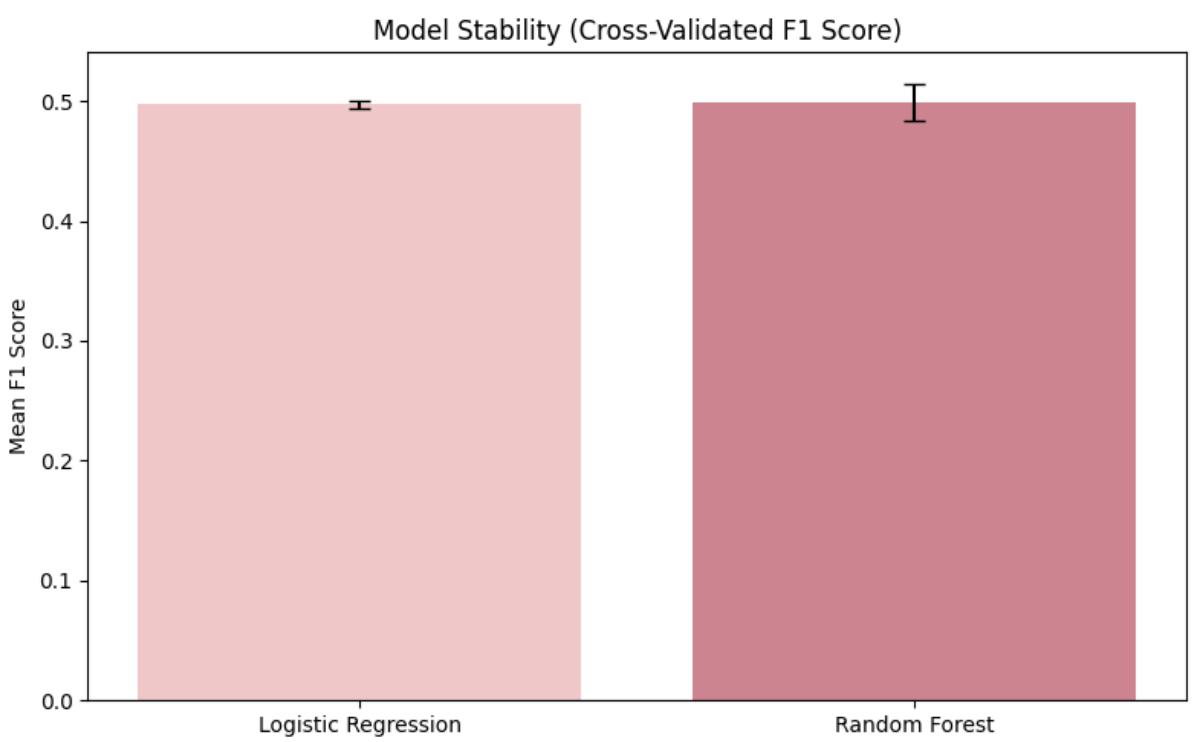
In [128…

```
# 15e. Cross-Validation Summary

cv_results_df = pd.DataFrame(cv_results)
if not cv_results_df.empty:
    display(cv_results_df[["Model", "CV Rows", "CV Mean F1", "CV Std F
else:
    print("Run at least one cross-validation cell before viewing the s
```

	Model	CV Rows	CV Mean F1	CV Std F1
1	Random Forest	30000	0.499005	0.015771
0	Logistic Regression	30000	0.496967	0.003352

In [129…

```
# 16. Plot Stability

if not cv_results_df.empty:
    plt.figure(figsize=(8, 5))
    plt.bar(cv_results_df["Model"], cv_results_df["CV Mean F1"], yerr=
    plt.title("Model Stability (Cross-Validated F1 Score)")
    plt.ylabel("Mean F1 Score")
    plt.tight_layout()
    plt.show()
else:
    print("No cross-validation results available to plot.")
```

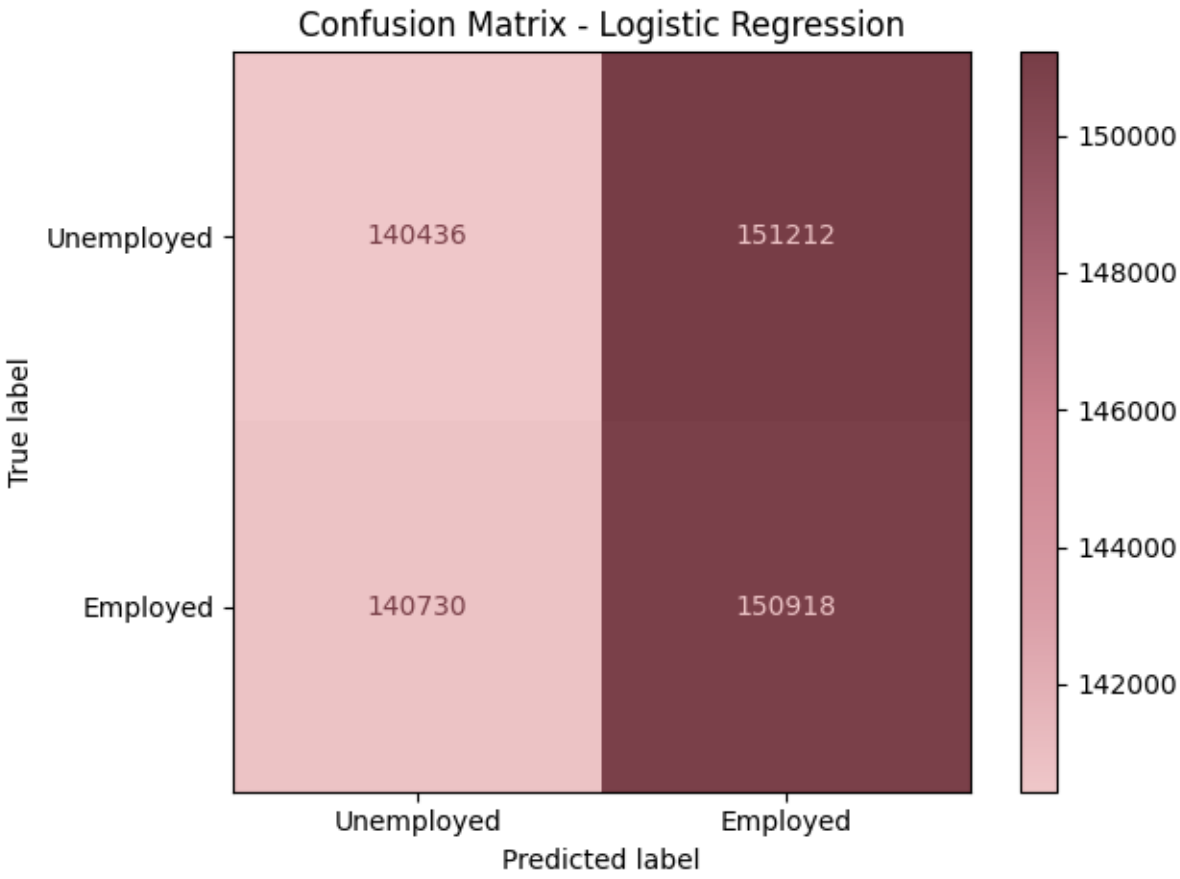


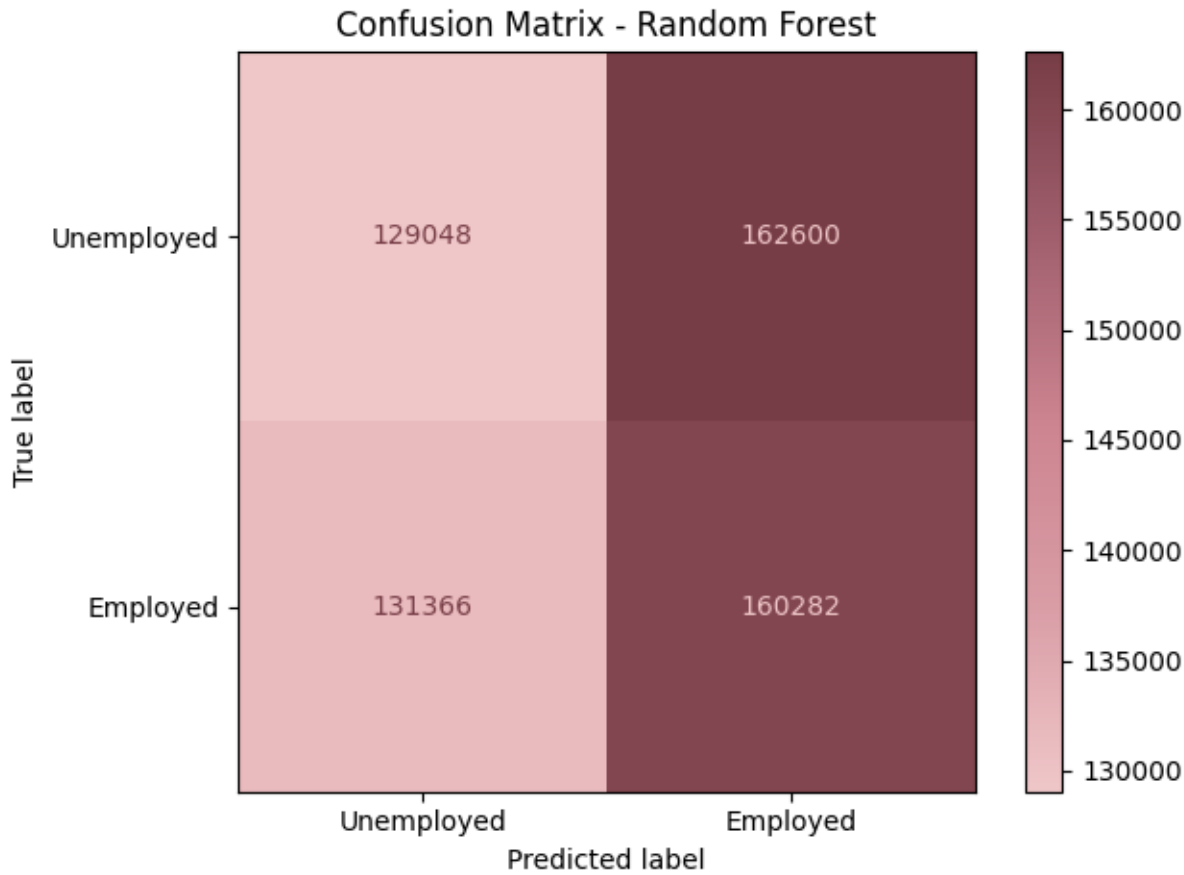
Diagnostics and Interpretation

In [130…

```
# 17. Confusion Matrices
```

```
from matplotlib.colors import LinearSegmentedColormap

pink_cmap = LinearSegmentedColormap.from_list(
    "custom_pink",
    ["#efc7c9", "#d99da6", "#cc8490", "#a15f6e", "#773d46"]
)

for model_name, pipeline in trained_pipelines.items():
    y_pred = pipeline.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)

    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=
    disp.plot(cmap=pink_cmap)
    plt.title(f"Confusion Matrix - {model_name}")
    plt.tight_layout()
    plt.show()
```





Feature Importance and Interpretation

In [131…

```
# 18. Feature Importance/Coefficients

logit_pipeline = trained_pipelines["Logistic Regression"]
rf_pipeline = trained_pipelines.get("Random Forest")

feature_names = logit_pipeline.named_steps["preprocessor"].get_feature

# Logistic Regression coefficients
logit_coefs = logit_pipeline.named_steps["classifier"].coef_[0]
logit_importance = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": logit_coefs,
    "AbsCoefficient": np.abs(logit_coefs)
}).sort_values(by="AbsCoefficient", ascending=False)

print("Top Logistic Regression Features:")
display(logit_importance.head(15))

# Random Forest feature importances
if rf_pipeline is not None:
    rf_importances = rf_pipeline.named_steps["classifier"].feature_imp
    rf_importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Importance": rf_importances
```

```
    }).sort_values(by="Importance", ascending=False)

    print("Top Random Forest Features:")
    display(rf_importance_df.head(15))
else:
    rf_importance_df = pd.DataFrame(columns=["Feature", "Importance"])
    print("Random Forest results are not available.")
```

```
Top Logistic Regression Features:
```

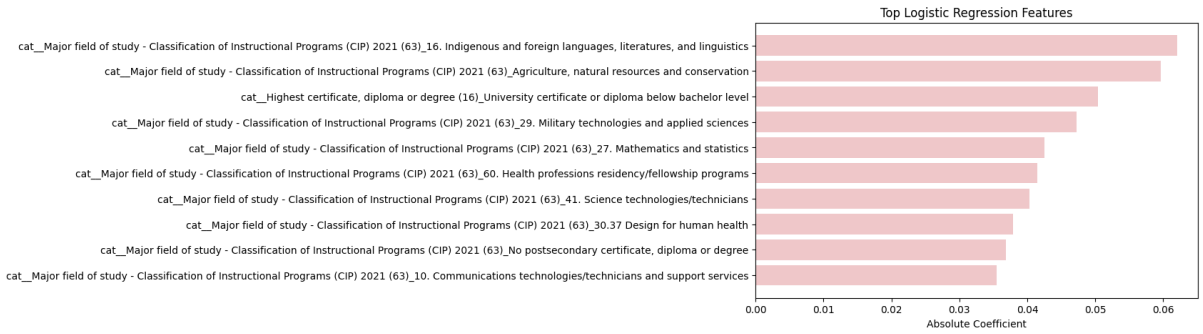
	Feature	Coefficient	AbsCoefficient
61	cat__Major field of study - Classification of ...	-0.062027	0.062027
99	cat__Major field of study - Classification of ...	-0.059674	0.059674
45	cat__Highest certificate, diploma or degree (1...	-0.050346	0.050346
70	cat__Major field of study - Classification of ...	0.047259	0.047259
68	cat__Major field of study - Classification of ...	0.042469	0.042469
97	cat__Major field of study - Classification of ...	-0.041444	0.041444
83	cat__Major field of study - Classification of ...	-0.040305	0.040305
74	cat__Major field of study - Classification of ...	-0.037909	0.037909
106	cat__Major field of study - Classification of ...	-0.036876	0.036876
55	cat__Major field of study - Classification of ...	0.035506	0.035506
63	cat__Major field of study - Classification of ...	0.033806	0.033806
82	cat__Major field of study - Classification of ...	0.033777	0.033777
43	cat__Highest certificate, diploma or degree (1...	0.033408	0.033408
109	cat__Major field of study - Classification of ...	0.032133	0.032133
50	cat__Major field of study - Classification of ...	-0.030565	0.030565

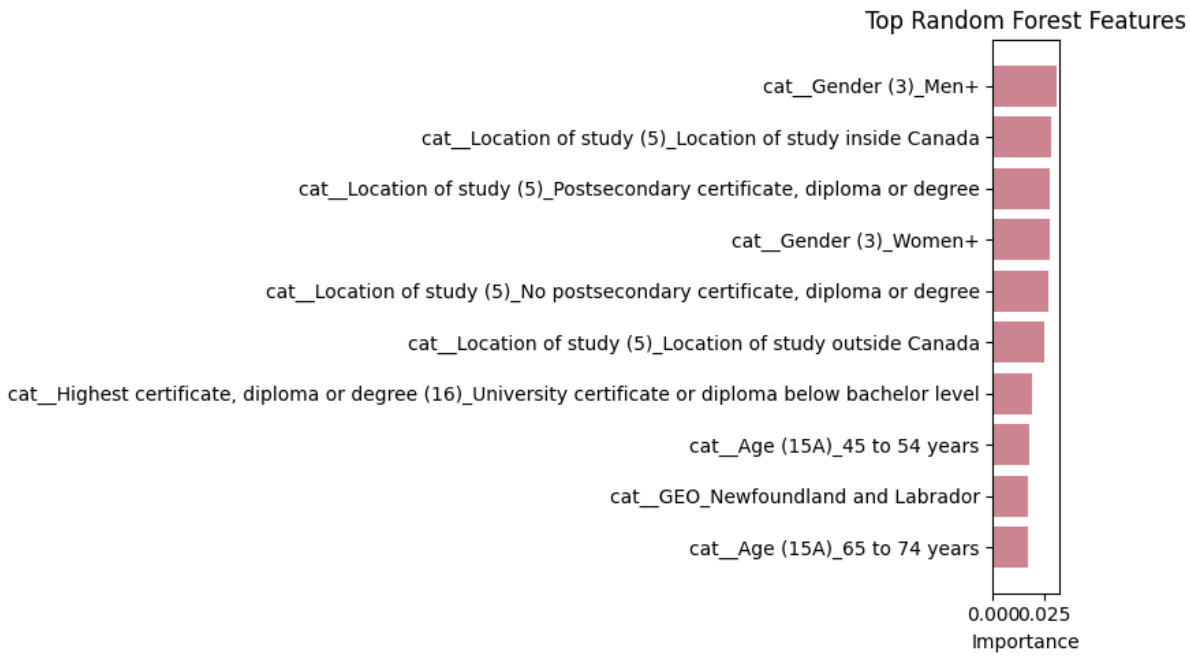
```
Top Random Forest Features:
```

	Feature	Importance
15	cat__Gender (3)_Men+	0.031014
46	cat__Location of study (5)_Location of study i...	0.028287
49	cat__Location of study (5)_Postsecondary certi...	0.028120
16	cat__Gender (3)_Women+	0.027799
48	cat__Location of study (5)_No postsecondary ce...	0.027468
47	cat__Location of study (5)_Location of study o...	0.025514
45	cat__Highest certificate, diploma or degree (1...	0.019082
26	cat__Age (15A)_45 to 54 years	0.017661
6	cat__GEO_Newfoundland and Labrador	0.017462
28	cat__Age (15A)_65 to 74 years	0.017002
30	cat__Age (15A)_75 years and over	0.016358
29	cat__Age (15A)_65 years and over	0.016292
20	cat__Age (15A)_25 to 29 years	0.015846
18	cat__Age (15A)_15 to 24 years	0.015542
24	cat__Age (15A)_30 to 34 years	0.015431

In [132…

```
# 19. Plot Features

top_n = 10

# Logistic Regression
top_logit = logit_importance.head(top_n).sort_values("AbsCoefficient")
plt.figure(figsize=(8, 5))
plt.barh(top_logit["Feature"], top_logit["AbsCoefficient"], color=["#e
plt.title("Top Logistic Regression Features")
plt.xlabel("Absolute Coefficient")
plt.tight_layout()
plt.show()

# Random Forest
if not rf_importance_df.empty:
    top_rf = rf_importance_df.head(top_n).sort_values("Importance")
    plt.figure(figsize=(8, 5))
    plt.barh(top_rf["Feature"], top_rf["Importance"], color=["#cc8490"
    plt.title("Top Random Forest Features")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.show()
```





In [133…

```
# 20. Save Results

results_df.to_csv("model_comparison_results.csv", index=False)
if not cv_results_df.empty:
    cv_results_df[["Model", "CV Rows", "CV Mean F1", "CV Std F1"]].to_
logit_importance.to_csv("logistic_feature_importance.csv", index=False
if not rf_importance_df.empty:
    rf_importance_df.to_csv("random_forest_feature_importance.csv", in

print("Results saved successfully.")
```

```
Results saved successfully.
```

Findings: Education and Employment

In [134…

```
# 21. Employment Rates by Education and Immigrant Status

# Identify immigrant columns
immigrant_cols = [
    "Immigrant status and period of immigration (11):Non-immigrants[2]
    "Immigrant status and period of immigration (11):Immigrants[3]"
]

# Keep relevant columns
```

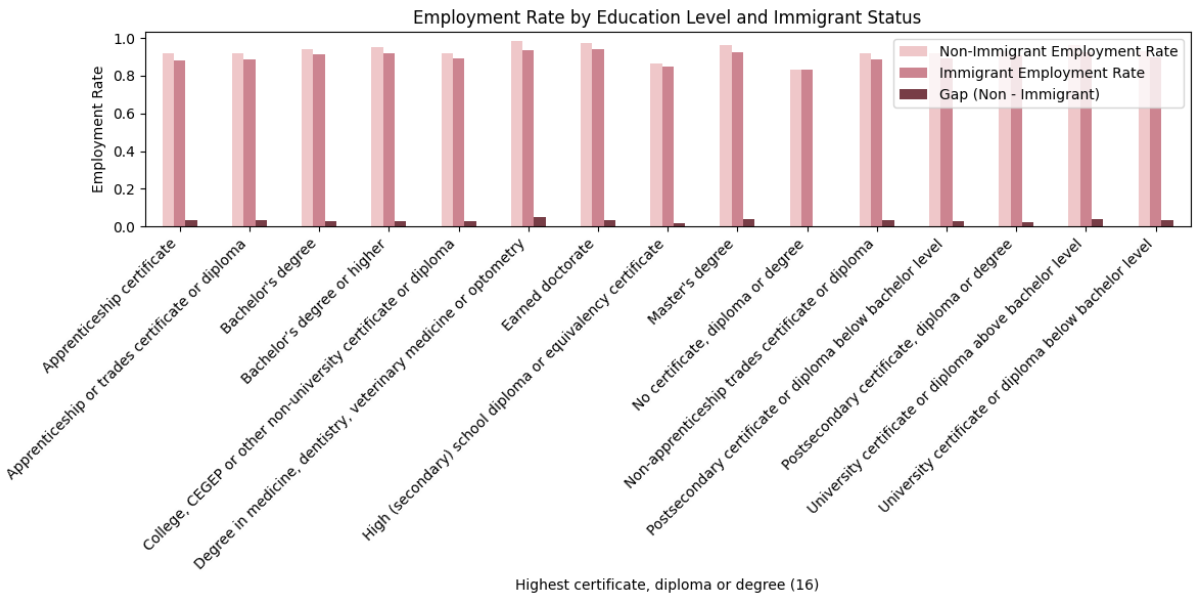
```
analysis_df = df_clean[
    ["Labour force status (8)",
     "Highest certificate, diploma or degree (16)"] + immigrant_cols
].copy()

# Filter to Employed / Unemployed only
analysis_df = analysis_df[
    analysis_df["Labour force status (8)"].isin(["Employed", "Unemploy
]

# Group by education + labour force status
grouped = analysis_df.groupby([
    "Highest certificate, diploma or degree (16)",
    "Labour force status (8)"
])[immigrant_cols].sum().reset_index()

# Pivot to get Employed vs Unemployed side by side
pivot = grouped.pivot(
    index="Highest certificate, diploma or degree (16)",
    columns="Labour force status (8)",
    values=immigrant_cols
)

# Calculate employment rates
employment_rates = pd.DataFrame({
    "Non-Immigrant Employment Rate":
        pivot[("Immigrant status and period of immigration (11):Non-im
        (pivot[("Immigrant status and period of immigration (11):Non-i
         pivot[("Immigrant status and period of immigration (11):Non-i

    "Immigrant Employment Rate":
        pivot[("Immigrant status and period of immigration (11):Immigr
        (pivot[("Immigrant status and period of immigration (11):Immig
         pivot[("Immigrant status and period of immigration (11):Immig
})

# Add Column for Gap between Non Immigrant and Immigrant
employment_rates["Gap (Non - Immigrant)"] = (
    employment_rates["Non-Immigrant Employment Rate"] -
    employment_rates["Immigrant Employment Rate"]
)

display(employment_rates)
```

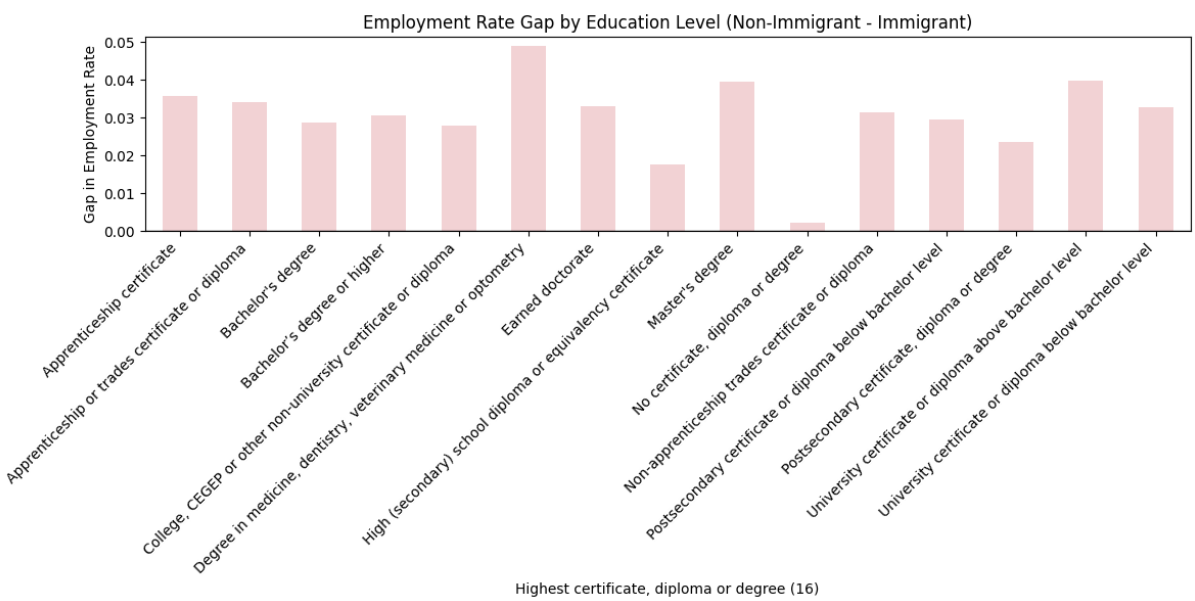
Highest certificate, diploma or degree (16)	Non-Immigrant Employment Rate	Immigrant Employment Rate	Gap (Non - Immigrant)
Apprenticeship certificate	0.917950	0.882318	0.035632
Apprenticeship or trades certificate or diploma	0.918268	0.884117	0.034151
Bachelor's degree	0.942520	0.913780	0.028741
Bachelor’s degree or higher	0.949045	0.918517	0.030528
College, CEGEP or other non-university certificate or diploma	0.917837	0.889788	0.028049
Degree in medicine, dentistry, veterinary medicine or optometry	0.981700	0.932777	0.048923
Earned doctorate	0.974903	0.941718	0.033184
High (secondary) school diploma or equivalency certificate	0.865276	0.847679	0.017596
Master's degree	0.964493	0.924933	0.039561
No certificate, diploma or degree	0.832849	0.830583	0.002266
Non-apprenticeship trades certificate or diploma	0.918959	0.887438	0.031521
Postsecondary certificate or diploma below bachelor level	0.918472	0.888864	0.029608
Postsecondary certificate, diploma or degree	0.931609	0.907919	0.023690
University certificate or diploma above bachelor level	0.963391	0.923453	0.039938
University certificate or diploma below bachelor level	0.928730	0.895904	0.032826

In [135…

```
# Plot Employment Rates
employment_rates.plot(kind="bar", figsize=(12,6), color=["#efc7c9", "#
plt.title("Employment Rate by Education Level and Immigrant Status")
plt.ylabel("Employment Rate")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# ["#efc7c9", "#d99da6", "#cc8490", "#a15f6e", "#773d46"]
```

```
employment_rates["Gap (Non - Immigrant)"].plot(
    kind="bar",
    figsize=(12,6),
    color=["#f2d2d4"]
)

#Plot Employment Rate Gap
plt.title("Employment Rate Gap by Education Level (Non-Immigrant - Imm
plt.ylabel("Gap in Employment Rate")
plt.axhline(0, color='black', linewidth=1)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```





Findings: Age and Employment

In [136…

```
# 22. Employment Rates by Age and Immigrant Status
```

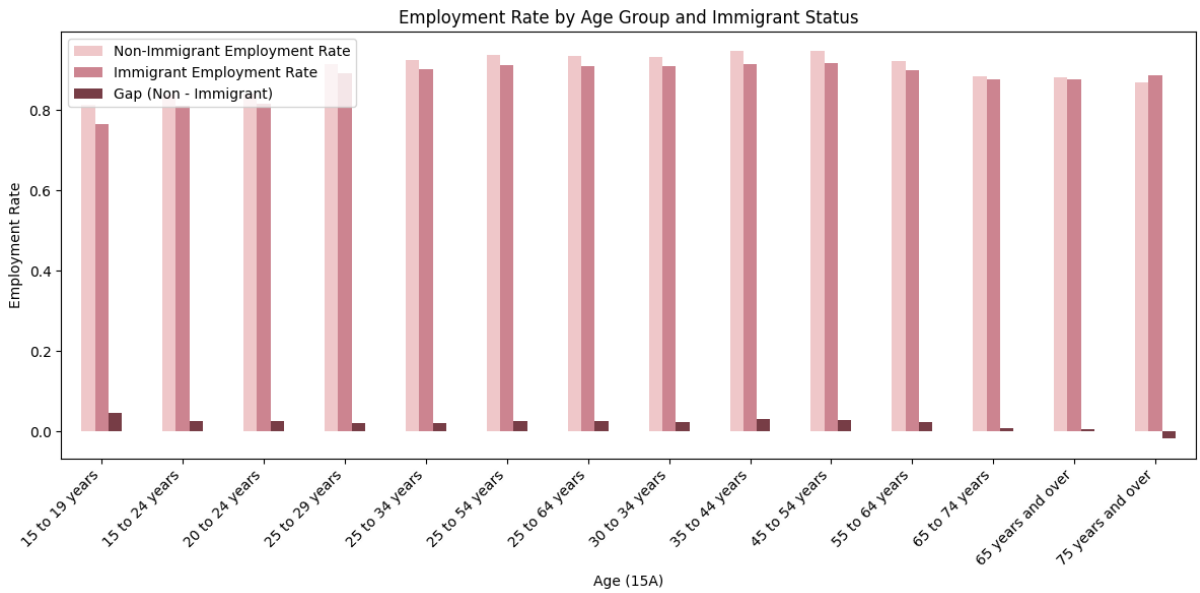
```
analysis_age = df_clean[
    ["Labour force status (8)", "Age (15A)"] + immigrant_cols
].copy()

analysis_age = analysis_age[
    analysis_age["Labour force status (8)"].isin(["Employed", "Unemplo
]

grouped_age = analysis_age.groupby(
    ["Age (15A)", "Labour force status (8)"]
)[immigrant_cols].sum().reset_index()

pivot_age = grouped_age.pivot(
    index="Age (15A)",
    columns="Labour force status (8)",
    values=immigrant_cols
)

# Calculate employment rates by age
age_rates = pd.DataFrame({
    "Non-Immigrant Employment Rate":
        pivot_age[("Immigrant status and period of immigration (11):No
        (pivot_age[("Immigrant status and period of immigration (11):N
         pivot_age[("Immigrant status and period of immigration (11):N

    "Immigrant Employment Rate":
        pivot_age[("Immigrant status and period of immigration (11):Im
        (pivot_age[("Immigrant status and period of immigration (11):I
         pivot_age[("Immigrant status and period of immigration (11):I
})

# Add Column for Gap between Non Immigrant and Immigrant
age_rates["Gap (Non - Immigrant)"] = (
    age_rates["Non-Immigrant Employment Rate"] -
    age_rates["Immigrant Employment Rate"]
)

display(age_rates)
```

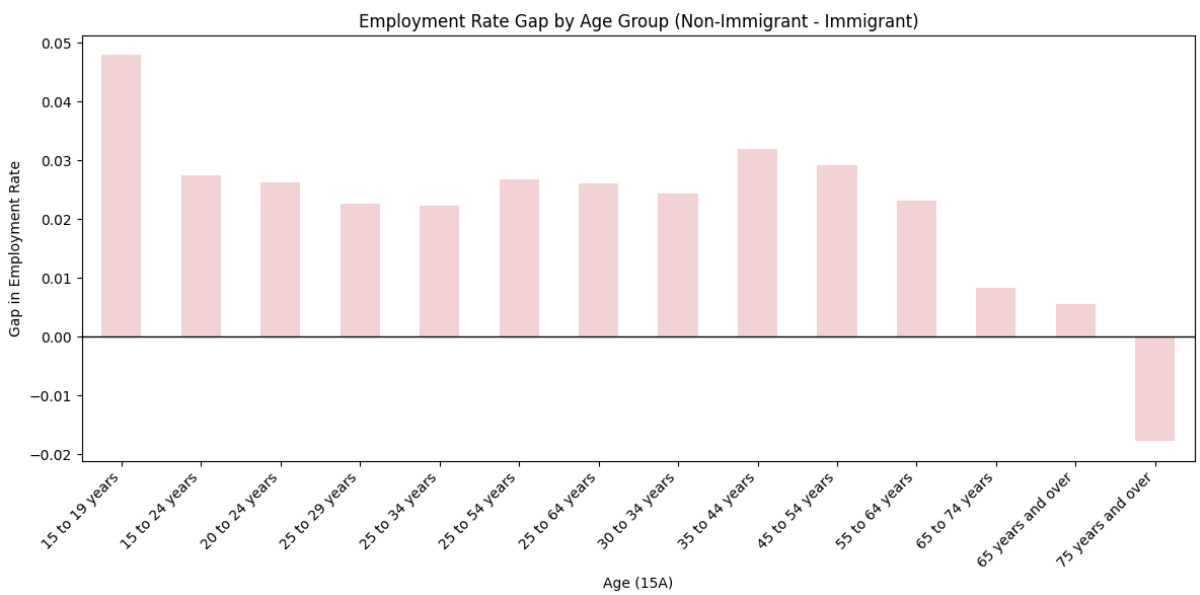
Age (15A)	Non-Immigrant Employment Rate	Immigrant Employment Rate	Gap (Non - Immigrant)
15 to 19 years	0.813528	0.765614	0.047914
15 to 24 years	0.837916	0.810398	0.027517
20 to 24 years	0.841911	0.815618	0.026294
25 to 29 years	0.915186	0.892613	0.022574
25 to 34 years	0.924178	0.901961	0.022217
25 to 54 years	0.938644	0.911853	0.026791
25 to 64 years	0.935384	0.909296	0.026088
30 to 34 years	0.933204	0.908933	0.024272
35 to 44 years	0.946704	0.914824	0.031879
45 to 54 years	0.946830	0.917673	0.029157
55 to 64 years	0.922762	0.899673	0.023089
65 to 74 years	0.885235	0.876887	0.008347
65 years and over	0.882637	0.877029	0.005608
75 years and over	0.870225	0.888039	-0.017814

In [137…

```
# Plot Employment Rates by Age

age_rates.plot(kind="bar", figsize=(12,6), color=["#efc7c9", "#cc8490"
plt.title("Employment Rate by Age Group and Immigrant Status")
plt.ylabel("Employment Rate")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
age_rates["Gap (Non - Immigrant)"].plot(
    kind="bar",
    figsize=(12,6),
    color="#f2d2d4"
)

plt.title("Employment Rate Gap by Age Group (Non-Immigrant - Immigrant
plt.ylabel("Gap in Employment Rate")
plt.axhline(0, color='black', linewidth=1)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```





Findings: Overall Employment Comparison

In [138…

```
# Overall Employment Rates by Immigrant Status

overall = df_clean[
```

```
    ["Labour force status (8)"] + immigrant_cols
].copy()

overall = overall[
    overall["Labour force status (8)"].isin(["Employed", "Unemployed"]
]

grouped_overall = overall.groupby("Labour force status (8)")[immigrant

non_emp = grouped_overall.loc["Employed", immigrant_cols[0]]
non_unemp = grouped_overall.loc["Unemployed", immigrant_cols[0]]

imm_emp = grouped_overall.loc["Employed", immigrant_cols[1]]
imm_unemp = grouped_overall.loc["Unemployed", immigrant_cols[1]]

overall_rates = pd.DataFrame({
    "Non-Immigrant Employment Rate": [non_emp / (non_emp + non_unemp)]
    "Immigrant Employment Rate": [imm_emp / (imm_emp + imm_unemp)]
})

display(overall_rates)
```

	Non-Immigrant Employment Rate	Immigrant Employment Rate
0	0.928128	0.90596